

# Technology Decision Reference — Staff+ Interview

One page per technology. The mechanism that drives the tradeoff, when to reach for it, when it flips, and the line to say in the room.

This sheet answers "**which technology and why**" — the axis a pattern-organized system design sheet doesn't cover. Every entry follows the same skeleton so it's scannable under pressure:

**mechanism** → **reach for / avoid** → **numbers** → **CAP** → **interview line** → **pushback**. The *interview line* is the choosing-not-pattern-matching signal; the *pushback* is where the obvious answer reverses, which is the staff tell.

**On the numbers:** these are interview *anchors* — order-of-magnitude figures to reason with out loud, not spec-sheet facts. Treat ranges as talking points, not exact values to be tested against.

## Contents

1. PostgreSQL — the default; ACID, joins, one node does more than you think
2. Cassandra — query-first, masterless, multi-region writes; you give up joins
3. DynamoDB — managed Dynamo-lineage KV; write-scaling with no cluster to run
4. Redis — in-memory speed; cache, counter, lock, queue, ephemeral state
5. Kafka — durable ordered log; decouple, buffer, replay, fan-out
6. Flink — stateful stream processing; windows, joins, exactly-once state
7. Message Queue (SQS / RabbitMQ) — per-message work dispatch; ack, retry, DLQ
8. S3 / Blob Storage — infinite cheap bytes; the bytes never touch your API
9. CDN — push bytes to the edge; the answer to "global" and "read-heavy static"
10. Vector DB — ANN over embeddings; the retrieval half of RAG
11. Workflow Orchestrator — durable fan-out/fan-in, exactly-once continuation
12. ZooKeeper / etcd — consensus-backed coordination; leader election, locks, config
13. Elasticsearch — inverted index; full-text + faceted search, not a primary store
14. Decision matrix — workload → pick, at a glance

## 1 · PostgreSQL

*The correct default. Reach for something else only when you can name the specific thing Postgres can't do at your scale.*

**Mechanism.** B-tree indexes, MVCC, single-leader replication. Crucially, Postgres does **not** update in place — an **UPDATE** writes a *new* row version and marks the old one dead, so readers get a consistent snapshot without blocking writers. **VACUUM** reclaims the dead versions (and is why bloat, HOT updates, and index write-amplification are things you tune). One primary takes writes; read replicas stream the WAL (write-ahead log), asynchronously by default. Write ceiling is set by local WAL **fsync** throughput and hot-row lock contention, not by replicas.

## Reach for it when

- You need **multi-row / multi-table ACID transactions** — money, inventory, multi-entity publish.
- Read patterns are **ad-hoc or unpredictable** — the planner + indexes handle new WHERE clauses for free.
- You need joins, aggregations, secondary indexes.
- Scale is anything below "genuinely global + huge." Which is almost always.

## Avoid / augment when

- Write volume genuinely exceeds one primary (see the trigger numbers below) → shard or LSM store.
- You need active-active **multi-region writes** — retrofitted, painful.
- Append-only firehose (metrics, events) — LSM fits better.
- **Connection storms** — each connection is ~a backend process (megabytes of RAM); a few thousand direct connections will exhaust it. Put **PgBouncer** (or equivalent) in front. This is the single most common real-world Postgres scaling gotcha, and it hits well before you run out of raw throughput.

**Triggers to break out (name the number).** The trigger is almost always a *specific hot table or row*, or a *write pattern* — not aggregate database size. 100M+ rows on one primary is routine.

- **< ~5k writes/s to one logical table** → you're fine on one primary. Don't move; you're solving a problem you don't have.
- **~5k–15k writes/s sustained** → **tune first**: batch inserts, offload the hot counter to Redis, faster-fsync disk (NVMe), fewer indexes on the hot table (HOT updates). Most "Postgres is slow" cases die here without an architecture change.
- **~15k–50k writes/s sustained to one table** → **shard Postgres** by a good key (app-level or Citus). You keep SQL and transactions *within* a shard; you give up cross-shard joins and painless rebalancing.
- **> ~50k writes/s sustained to one table, append-only firehose (metrics/events), or active-active multi-region writes** → move that hot table to an **LSM / wide-column store** (Cassandra / DynamoDB) or a purpose-built TSDB. Here the *write pattern*, not the size, forces the change.
- **A single hot row taking > ~500–1k serialized updates/s** (a global counter, one inventory row) → the ceiling is row-lock contention, not table throughput. Offload *that one value* to Redis ( **INCR** ) or shard the counter; the rest of the table stays in Postgres.

## Durability layers (know cold)

- **Local WAL fsync** survives a *crash* (power loss, panic) — recovery replays it. It does **not** survive losing the disk/node.
- **Synchronous replication** ( **ANY 1** of several standbys, ideally another AZ) means no acknowledged write lives on a single disk. Use **ANY 1** quorum-with-slack, not a single named standby — requiring a specific replica means its failure blocks all writes.
- **WAL archiving to object storage + PITR** survives *logical* corruption (a bad **DELETE** replicates faithfully to every replica; only point-in-time recovery saves you).

## Numbers to anchor

| Point read (indexed)       | < 1 ms                         |
|----------------------------|--------------------------------|
| Writes/s, one primary      | ~5k–15k                        |
| Shard trigger (one table)  | ~15k–50k writes/s              |
| LSM/firehose trigger       | > ~50k writes/s or append-only |
| Serialized updates/hot row | ~500–1k/s                      |
| Rows before sharding hurts | ~10–100M+                      |
| Connections                | pool it (~hundreds direct max) |

**CAP / consistency.** CP-leaning, but really *ACID on one node + tunable replication*. Transactional consistency (isolation up to SERIALIZABLE) is a native capability. Replication consistency is a separate knob: async replicas are eventually consistent; `synchronous_commit = remote_apply` or reading the primary gives strong.

**Interview line.** I default to Postgres and make something else justify itself — transactions, joins, and query flexibility for free, one primary does 5–15k writes/s, reads scale on replicas. I only move off it when I can name the exact property it can't provide at my scale — a specific hot table past ~15–50k writes/s, a firehose, or a multi-region write — and I pool connections and tune the hot table before I touch the architecture.

**Pushback / when it flips.** "Postgres doesn't scale" is usually wrong — a **single hot row**, a **multi-region write**, or **connection exhaustion** flips it, not raw size. Sharded Postgres by a good key gets most of Cassandra's write distribution; what you lose is automatic rebalancing and painless geo-replication, not the sharding itself.

## 2 · Cassandra

*Query-first, masterless, born multi-region. You trade joins and ad-hoc queries for cheap global replication and painless rebalancing.*

**Mechanism.** LSM tree + consistent hashing + masterless replication. Writes append to a memtable + commitlog, flush to immutable SSTables — sequential I/O, so the write ceiling is high. Compaction merges SSTables in the background. Consistent hashing (with vnodes) places partitions on a ring — adding a node auto-rebalances token ranges. **Any node can coordinate a write** and forward to the replica set; there's no special primary. Consistency is tunable per query ( `ONE` , `QUORUM` , `ALL` ); multi-datacenter replication is a config declaration.

### Reach for it when

- Access is **known, key-based lookups** — "everything for this id," "this partition sorted by time."
- Global reads needing **local-region latency** — multi-DC replication out of the box.
- Very high write throughput, append-heavy.
- You want capacity-by-adding-nodes with no manual reshard.

### The model tax (know cold)

- **No joins, no ad-hoc queries.** Query by non-partition column = full scan.
- **One table per query** — denormalize, dual-write, own consistency across copies yourself.

- No general transactions. LWT (Paxos) does single-partition compare-and-set only, and it's slow.
- Incomplete clustering key **silently upserts** (clobbers rows).
- **Tombstones**: deletes don't remove data, they write a marker that lingers until compaction. Read across many tombstones and latency craters — this is why Cassandra is a *terrible* queue (write-then-delete churns tombstones). The #1 footgun after clustering-key upserts.

### Numbers to anchor

| Write throughput      | very high, scales out     |
|-----------------------|---------------------------|
| Single-partition read | low ms                    |
| Add capacity          | join a node               |
| Consistency           | tunable per query         |
| $R + W > N$           | → strong-consistent reads |

**CAP / consistency.** AP by design, **tunable toward CP**. **QUORUM** reads + **QUORUM** writes satisfy  $R + W > N$ , so the read overlaps the latest write → strong consistency, at the cost of latency and availability during partitions. Even at **ALL**, you get *replication* consistency, never *transactional* — no "update these three rows or none."

**Interview line.** Cassandra fits keyed lookups read globally and written rarely. Its edge over sharded Postgres isn't write throughput at my volume — it's that consistent hashing makes rebalancing automatic and multi-region replication a config line, not a project. I accept no joins and eventual consistency (tunable to strong via QUORUM/QUORUM), which cost nothing for this access pattern.

**Pushback / when it flips.** Below genuine global scale, **Postgres is the better default** even for keyed data — you keep transactions and query flexibility and defer the operational tax (repair, compaction, tombstone tuning, eventual-consistency reasoning). Cassandra is right when scale × geo-distribution × access-pattern-simplicity all hold at once. And never model a queue or high-churn delete workload on it — tombstones will punish you.

## 3 · DynamoDB

*The managed, serverless member of the Dynamo family. You buy Cassandra-style write-scaling and multi-region replication without running a cluster — and pay in partition-key discipline and per-request pricing.*

**Mechanism.** The **partition key** is hashed to place an item on a partition; an optional **sort key** orders items within a partition. Storage auto-shards as data and throughput grow — no nodes to manage. Reads default to **eventually consistent** (cheaper, served from any replica); you opt into **strongly consistent** reads per request (more cost, leader replica, single-region only). Capacity is **on-demand** (pay-per-request, auto-scales to spikes) or **provisioned** (RCU/WCU you set, cheaper at steady load). Secondary indexes: **GSI** (different partition key, its own throughput, eventually consistent) and **LSI** (same partition key, alternate sort key, fixed at table creation). **DynamoDB Streams** emits an ordered per-item change log — CDC for triggers, replication, and fan-out.

**Reach for it when**

- Known **key-based access at scale** and you want **zero ops** — no cluster to run, patch, or rebalance.
- **Serverless / spiky** workloads — on-demand capacity tracks traffic without a capacity plan.
- Predictable **single-digit-ms** latency at any scale for keyed lookups.
- Multi-region **active-active** (global tables) as a config option, not a project.

### The model tax (know cold)

- **Hot partition**: throughput is spread across partitions by key. A skewed key (one celebrity, one tenant) concentrates load on a single partition and throttles — *even when the table has spare total capacity*. Same failure class as Cassandra's hot partition; pick a high-cardinality, evenly-accessed partition key.
- **Item size cap 400 KB** — large blobs go to S3 with the key stored in Dynamo.
- **Single-table design** — model access patterns up front and overload one table with composite keys; ad-hoc queries aren't a thing.
- **No joins; transactions are limited** — `TransactWriteItems` exists but caps at 25 items and costs ~2× throughput. Not a substitute for relational transactions.
- **Cost surprises** — `Scan` and unbounded GSIs get expensive fast. Model your reads as key lookups; never design around scanning.

### Numbers to anchor

| Latency (keyed) | single-digit ms                       |
|-----------------|---------------------------------------|
| Item size cap   | 400 KB                                |
| Read default    | eventual (strong opt-in, per request) |
| Transaction     | ≤ 25 items, ~2× cost                  |
| Scale           | auto-shard, no ops                    |

**CAP / consistency.** AP by default (eventually consistent reads), **CP per request** via strongly-consistent reads within a single region. Global tables are **last-writer-wins**, eventually consistent across regions — no cross-region strong consistency. Like Cassandra: tunable read consistency, never cheap multi-partition transactional consistency.

**Interview line.** DynamoDB is the managed Dynamo-lineage store: partition-key scaling and global replication with no cluster to operate, in exchange for designing every access pattern into the key up front. Reads are eventually consistent unless I ask for strong per request, and my one real job is a high-cardinality partition key so I don't hot-spot one partition while the table looks under-utilized.

**Pushback / when it flips.** Below real scale, Postgres still wins — you keep joins, transactions, and the ad-hoc queries Dynamo's single-table model forbids. DynamoDB vs Cassandra is mostly **buy vs run**: same data model and tradeoffs, DynamoDB is managed/serverless with per-request cost, Cassandra is self-hosted with fixed cluster cost. The classic incident is the **hot partition** — a skewed key throttles one partition while the table looks under-provisioned.

## 4 · Redis

*In-memory, microsecond ops. A Swiss-army knife: cache, counter, lock, rate limiter, ephemeral queue, leaderboard. Rarely your source of truth.*

**Mechanism.** RAM-resident data structures with **single-threaded command execution** — every command (and every Lua script) runs atomically with no locks, which is the load-bearing property. (Modern Redis uses I/O threads for network, but execution of the actual commands is still serialized, so the atomicity guarantee holds.) Durability is optional: **RDB** snapshots + **AOF** append log (**everysec** is the usual). Replicas + Sentinel/Cluster for HA. Treat it as a fast cache that **can** persist, not a database that happens to be fast.

### Reach for it when

- **Cache** in front of a slower store (the default use).
- **Atomic counters** — **INCR** / **DECR** for rate limits, inventory admission, fan-in completion.
- **Distributed lock** (short-lived; **SET NX PX** + fencing token).
- **ZSET** for leaderboards, waiting-room ordering, timer queues.
- Ephemeral state with **TTL auto-expiry** (sessions, holds).

### Cache patterns to name

- **Cache-aside** (lazy): app checks cache, on miss reads DB and populates. The default. Risk: stale entries → set TTLs / invalidate on write.
- **Write-through / write-behind**: write hits cache (and DB sync/async). Stronger freshness, more coupling.
- **Stampede / thundering herd**: a hot key expires and thousands of requests hit the DB simultaneously to refill. Mitigate with a short lock around the refill (one request repopulates), request coalescing, or probabilistic early expiry. Directly relevant to any "hot cached value" design.

### Avoid / careful when

- It's your **only** copy of important data — a failover to a lagging replica loses recent writes.
- Dataset > RAM. It's memory-bound; that's the cost model.
- TTL as a business trigger: expiry **deletes a key**, it doesn't run your compensation. Keyspace notifications are at-most-once — don't rely on them for correctness.

### Numbers to anchor

| Op latency           | ~0.1–0.5 ms   |
|----------------------|---------------|
| Throughput, one node | ~100k+ ops/s  |
| Bound                | RAM + network |

**CAP / consistency.** CP within a shard for single-key ops (atomic execution), but the *durability/HA* story is weak by default — async replication + Sentinel failover can lose recently-acknowledged writes. Redis Cluster shards by key; multi-key atomicity only within a hash slot.

**Interview line.** I reach for Redis when I need one thing very fast and can tolerate losing it: a cache, an atomic counter, a TTL'd hold. Serialized command execution means every op and every Lua script is atomic, which is why it's the right place for admission control and rate limiting. I keep the durable truth in Postgres behind it, and I plan for cache stampede on hot keys.

**Pushback / when it flips.** For a hold/expiry pattern, Redis TTL only wins if the **key IS the hold** (its disappearance is the release). If Postgres owns the counter, a DB sweeper beats Redis notifications — durable and self-healing. Use Redis for the ticket-onsale scale (100k holds/s, visible countdown); keep the sweeper for the hotel scale.

## 5 · Kafka

*A durable, ordered, replayable log. Not a queue you drain — a commit log consumers read at their own offset. The backbone for decoupling and buffering.*

**Mechanism.** Partitioned append-only log; consumers track offsets. Each topic splits into partitions; order is guaranteed **within** a partition, not across. Messages persist for a retention window — consumers replay by rewinding their offset. Producers key messages to control partition (same key → same partition → ordered). Delivery is **at-least-once** by default; consumers must be idempotent.

### Reach for it when

- **Decouple** producers from consumers — upload service emits, transcode fleet consumes independently.
- **Buffer a firehose** — absorb spikes, consumers process at their own rate (load leveling).
- **Fan-out** one event to many consumer groups (analytics + search index + notifications).
- **Event sourcing / CDC** — the log is the source of truth; replay to rebuild.

### Operational realities (know cold)

- **Partition count is your parallelism ceiling:** within a consumer group, consumers  $\leq$  partitions (extra consumers sit idle). You pick partition count up front and it's awkward to reduce — size for peak parallelism.
- **Consumer-group rebalancing:** when a consumer joins/leaves, partitions are reassigned and processing pauses briefly ("stop-the-world"). Frequent rebalances (flapping consumers, long processing) are a classic Kafka incident.
- **Ordering vs. parallelism tension:** you get order *within* a partition, so more partitions = more parallelism but coarser ordering. Key by the entity whose order matters.

### Avoid / careful when

- You need **per-message ack/delete**, priority, or delay — that's a task queue (see the queue page), not a log.
- You need **global ordering** — you only get per-partition.
- Low-volume simple job dispatch — Kafka is operational weight you may not need.

### Numbers to anchor

| Throughput  | very high (MB/s+/partition)   |
|-------------|-------------------------------|
| Ordering    | per-partition only            |
| Delivery    | $\geq$ once (make idempotent) |
| Parallelism | $\leq$ partition count        |
| Retention   | hours→forever, configurable   |



**CAP / consistency.** CP-leaning: a partition has a leader + ISR (in-sync replicas); `acks=all` waits for the ISR, trading latency for no-loss. Exactly-once semantics (EOS) exist **but only within Kafka** — read-process-write across Kafka topics. It does **not** extend exactly-once to external side effects (your DB, an API), which still need idempotency.

**Interview line.** I use Kafka to decouple and buffer: producers append, consumer groups read independently at their own offset, and retention lets me replay to rebuild a downstream or add a consumer. I key by entity for per-partition ordering, size partitions for peak consumer parallelism, and make consumers idempotent because delivery is at-least-once — Kafka's exactly-once doesn't cover my database write.

**Pushback / when it flips.** Reaching for Kafka on a simple job queue is over-engineering — if you want per-message ack, retry, priority, and a DLQ without replay or fan-out, a **task queue is simpler**. Kafka earns its operational cost when you need replay, multiple independent consumers, or firehose buffering — name which one. And Kafka only *transports* the stream; the moment you need to window, aggregate, or join it, that's Flink's job, not a hand-rolled consumer.

---

## 6 · Flink

*Stateful stream processing with event-time windows and exactly-once state. Kafka moves the events; Flink computes over them — running aggregations, joins, and windows on the stream itself.*

**Mechanism.** A dataflow graph of operators runs continuously over unbounded streams. Operators hold **keyed state** (per-key aggregates, join buffers) in a local **state backend** — RocksDB on local disk when state is large. Fault tolerance is **distributed checkpointing**: Chandy–Lamport barriers flow through the graph and snapshot each operator's state consistently; on failure the job rewinds to the last checkpoint and replays from the source offset → **exactly-once state**. **Event-time** processing with **watermarks** handles out-of-order and late events: a watermark asserts "no more events older than T," which fires time windows correctly regardless of arrival order — versus **processing-time** (wall clock, simpler, but wrong under lateness).

### Reach for it when

- **Streaming aggregations / windows** — per-minute counts, sliding-window rates, sessionization.
- **Stream joins** and enrichment — join two topics, or a stream against a slowly-changing table.
- **Real-time** analytics / alerting where batch latency is too slow.
- Stateful **complex-event processing** (pattern detection) over a firehose.

### The distinctions to say

- **Kafka vs Flink**: Kafka is transport + storage (the log); Flink is compute over it. "Kafka moves and retains; Flink windows, aggregates, and joins." **Kafka Streams** is the lighter embedded library for simpler per-app transforms.
- **Event-time vs processing-time**: event-time + watermarks is correct under out-of-order / late data; processing-time is simpler but wrong when events arrive late. Naming watermarks is the signal you actually understand streaming.
- **Stream vs batch**: Flink treats batch as a bounded stream; reach for it when you need continuous low-latency results, not a nightly job.

### Avoid / careful when



- Simple **stateless** transform or fan-out — Kafka consumers / Kafka Streams are lighter.
- Small volume or an occasional job — a cron batch is far less operational weight.
- Large keyed state is real ops: state-backend sizing, checkpoint tuning, savepoint / restore on redeploy.

### Numbers to anchor

| Latency    | ms → sub-second                         |
|------------|-----------------------------------------|
| Guarantee  | exactly-once <i>state</i> (checkpoints) |
| Time model | event-time + watermarks                 |
| State      | local + checkpointed (RocksDB)          |

**CAP / consistency.** Gives **exactly-once state** internally via checkpoint-and-replay; **end-to-end** exactly-once needs a **transactional / idempotent sink** (two-phase-commit connector) or the output is at-least-once. Same caveat as Kafka EOS: the guarantee holds *within* the processing boundary; external side effects still need idempotency.

**Interview line.** When I need to compute over a stream — windowed aggregations, joins, sessionization in real time — I reach for Flink, not a Kafka consumer with a hand-rolled counter. It keeps large keyed state locally, checkpoints it for exactly-once recovery, and uses event-time watermarks so late events still land in the right window. End-to-end exactly-once still needs an idempotent sink.

**Pushback / when it flips.** Flink is real operational weight — state backends, checkpoints, savepoints. If the transform is stateless or the window is trivial, Kafka Streams or plain consumers are simpler. It earns its cost only when you have large, long-lived, keyed state and need correctness under out-of-order events — name the window and the state, or you don't need Flink.

## 7 · Message Queue (SQS / RabbitMQ)

*Per-message work dispatch with ack, redelivery, and dead-lettering. The "distribute tasks to workers" primitive — distinct from Kafka's replayable log.*

**Mechanism.** A broker holds messages; workers pull (or are pushed) one, process it, and **ack** to delete it. No ack (worker crash/timeout) → the message becomes visible again and is redelivered. After N failed attempts it goes to a **dead-letter queue (DLQ)** for inspection. Unlike a log, a consumed+acked message is *gone* — there's no offset to rewind, no replay, no second consumer group reading the same message.

### Reach for it when

- **Task/job dispatch** to a worker pool — resize emails, thumbnails, webhooks, background jobs.
- You want **per-message retry + DLQ** without building it (the broker owns redelivery).
- **Priority** or **delayed/scheduled** delivery (native in RabbitMQ / SQS).
- Decoupling where you *don't* need replay or fan-out — competing consumers draining one queue.

**Log vs. queue — the distinction to say**

- **Kafka (log)**: retained, replayable, multiple independent consumer groups, ordering per partition. "Many readers, rewindable history."
- **Queue (SQS/RabbitMQ)**: consumed-and-deleted, per-message ack/retry/DLQ, work distributed across competing consumers. "One logical reader, work drains."
- Rule of thumb: **need replay or multiple consumers of the same event** → **log**. **Need work distribution with retry/DLQ** → **queue**.

#### Avoid / careful when

- You need to **replay** history or add a new consumer that sees past events — that's a log.
- **Strict global ordering** — most queues don't guarantee it (SQS standard is best-effort; SQS FIFO trades throughput for order).
- Very high fan-out to many independent subscribers — pub/sub or a log fits better.

#### Numbers to anchor

| Delivery | ≥ once (FIFO variants: exactly-once-ish) |
|----------|------------------------------------------|
| Ordering | best-effort (FIFO opt-in)                |
| Retry    | built-in (visibility timeout)            |
| DLQ      | built-in after N attempts                |
| Replay   | ✗ (consumed = gone)                      |

**CAP / consistency.** Managed queues (SQS) prioritize availability and at-least-once delivery; ordering and dedup are opt-in (FIFO) at a throughput cost. Design consumers to be **idempotent** regardless — redelivery is normal, not exceptional.

**Interview line.** For distributing background work to a pool with automatic retry and dead-lettering, I use a task queue, not Kafka. Workers ack on success; a crash just redelivers after the visibility timeout, and poison messages fall to a DLQ. I reach for Kafka instead only when I need replay or multiple independent consumers of the same stream.

**Pushback / when it flips.** The mistake is defaulting to Kafka for everything "async." If there's one logical consumer draining work and you want retry/DLQ semantics for free, a queue is simpler and cheaper to operate. Flip to a log the moment you need replay, event fan-out, or an audit trail of what happened.

## 8 · S3 / Blob Storage

*Effectively infinite, cheap, durable bytes over HTTP. The rule: large binaries live here, your DB stores the pointer, and the bytes never transit your API tier.*

**Mechanism.** Key→object store over HTTP, ~11-nines durability. Not a filesystem — flat keyspace, objects are immutable blobs (overwrite, don't edit). Standard classes replicate across ≥3 AZs (One Zone classes deliberately don't — cheaper, less durable). Tiered storage (hot → infrequent → archive) trades retrieval latency for cost. **Presigned URLs** let clients read/write directly with time-limited credentials — your server signs, the client transfers. **Multipart upload** splits large objects into parts (parallel, resumable).

## Reach for it when

- Any **large binary**: video, images, uploads, backups, ML artifacts, logs.
- Static assets to serve via CDN (S3 as origin).
- Data-lake / event archive (cheap, queryable later via Athena/etc).
- Decoupling upload transfer from your app servers entirely.

## The two patterns to say

- **Presigned direct upload** — client PUTs straight to S3; API only issues URLs. Bytes bypass your tier.
- **Multipart** for big files — one presigned URL per part, parallel, per-part retry, resumable via `ListParts`. Client picks part size subject to 5 MB min / 10k parts max.

## Numbers to anchor

| Durability               | ~11 nines (standard, multi-AZ) |
|--------------------------|--------------------------------|
| Single PUT cap (S3)      | 5 GB                           |
| Multipart part min / max | 5 MB / 10k parts               |
| First-byte latency       | tens of ms                     |

**CAP / consistency.** **Strongly read-after-write consistent** for all operations since Dec 2020 — a GET after a PUT returns the latest object. What it does *not* have: cross-object transactions, conditional multi-key atomicity, or querying inside objects. It's a durable key→blob store, not a database.

**Interview line.** Large binaries go to blob storage and the DB keeps only the key. Clients upload via presigned URLs so gigabytes never touch my API tier, and anything big uses multipart — one URL per part, parallel, resumable. I front it with a CDN for reads and drive state transitions off object-created events rather than trusting the client's "done."

**Pushback / when it flips.** S3 is now strongly consistent, so don't cite the old eventual-consistency caveat — the real limits are no transactions and no querying inside objects. Don't store data you'll filter/join on as blobs. And it's the **pointer-in-DB** pattern: an object with no DB row is an orphan you pay for — reconcile with a sweeper.

---

## 9 · CDN

*Cache bytes at edge PoPs close to users. The answer to "global users" and "read-heavy static/cacheable content." Turns 20k QPS for one object into ~one origin fetch per PoP.*

**Mechanism.** Geographically distributed edge caches. User hits the nearest PoP; on a cache hit it serves locally (low latency, zero origin load). On a miss it fetches from origin, caches per TTL/headers, serves. `Cache-Control` / `ETag` govern freshness. Collapses read fan-out: N users pulling the same object = ~1 origin fetch per PoP per TTL. Also absorbs traffic spikes and DDoS at the edge.

## Reach for it when

- **Static assets**: JS/CSS/images, video segments, downloads.

- **Global read audience** for the same content.
- A **hot cacheable value** read enormously — e.g. the waiting-room "admitted" counter cached 2s collapses 500k polls to near-zero origin.
- Shielding origin from spikes.

#### Avoid / careful when

- **Personalized / per-user** responses — low hit rate, little benefit (edge compute is the escape hatch).
- Highly dynamic data where staleness is unacceptable.
- Invalidation is the hard part — **short TTL** (let it expire) beats active purge where you can tolerate seconds of staleness.

#### Numbers to anchor

| Edge hit latency | single-digit→tens ms |
|------------------|----------------------|
| Origin offload   | often > 95%          |
| Invalidation     | TTL > active purge   |

**CAP / consistency.** Deliberately eventually consistent — the edge serves possibly-stale content bounded by TTL. That staleness *is* the tradeoff you accept for latency and origin offload; design around "what's cacheable and for how long," not "how do I make the edge fresh."

**Interview line.** A CDN pushes cacheable bytes to edge PoPs so global reads hit a nearby cache instead of my origin — that's how one popular object survives 20k QPS at ~one origin fetch per PoP. I lean on short TTLs over active invalidation whenever seconds of staleness are acceptable, which is most static and near-static content.

**Pushback / when it flips.** A CDN does nothing for **personalized or write** paths — don't wave it at a dynamic problem. The real design question is always **invalidation**: reason about what's cacheable and for how long. Staleness tolerance is the actual variable.

## 10 · Vector Database

*Approximate nearest-neighbor search over embedding vectors. The retrieval half of RAG: "find the k chunks most semantically similar to this query."*

**Mechanism.** ANN index over high-dim vectors. Text/images → embeddings (e.g. 768–3072 dims) via a model. Query is embedded the same way; the DB finds nearest neighbors by cosine/dot distance. Exact search is O(N) per query, so it uses an **approximate** index — usually **HNSW** (navigable small-world graph): logarithmic-ish search, tunable recall vs. latency. Metadata filtering (tenant, date, ACL) runs alongside vector search — critical and easy to get wrong.

#### Reach for it when

- **RAG**: ground an LLM in your corpus — retrieve top-k chunks, stuff into context.
- Semantic search (meaning, not keywords).
- Recommendations / dedup / similarity by embedding.

**Pipeline to say.** Ingest: chunk → embed → upsert with metadata. Query: embed → ANN top-k → filter by ACL → rerank → into prompt. **Chunking strategy and reranking** move quality more than the DB choice.

### Avoid / careful when

- Keyword/exact-match is what users want — **BM25/Elasticsearch** may beat or hybridize with vectors.
- Small corpus — a library (FAISS) or **pgvector** in Postgres beats a new system.
- **Permission leakage**: embeddings ignore ACLs. Filter by permission at query time or you retrieve across tenants.

### Numbers to anchor

| Embedding dims  | ~768–3072                                |
|-----------------|------------------------------------------|
| Raw vector size | dims × 4 B (float32)                     |
| Quantization    | int8 ~4×, binary ~32× smaller            |
| 1M × 1536-dim   | ~6 GB raw + HNSW graph (often > vectors) |
| Index           | HNSW (recall↔latency)                    |

**CAP / consistency.** Usually eventually consistent and read-optimized; a just-upserted vector may not be immediately searchable. Fine for RAG (corpus changes slowly). The deployed **memory footprint** is the real gotcha — the HNSW graph frequently exceeds the raw vectors, and quantization is how you fit large corpora in RAM.

**Interview line.** For RAG I embed and index chunks with HNSW, then at query time embed the question, pull top-k by cosine, filter by the user's ACL, and rerank before building the prompt. Chunking and reranking drive quality more than which vector DB. Below a few million vectors I'd just use pgvector; above that I'd budget for the index memory and consider quantization.

**Pushback / when it flips.** Vector search isn't always the answer — **hybrid** (BM25 + vector) often wins because pure semantic misses exact terms (names, IDs, codes). And the sharp failure mode is **permissions**: sensitive data in embeddings/logs and cross-tenant retrieval. Raise ACL-at-query-time unprompted — it's the People-Innovation kill-shot.

---

## 11 · Workflow Orchestrator

*Temporal / Step Functions. Durable multi-step workflows with fan-out/fan-in and exactly-once continuation. The answer to "coordinate N async tasks and know when all are done."*

**Mechanism.** Event-sourced state machine + replay. Every state transition is appended to a durable history log — that log is truth, not in-memory state. On crash, the workflow re-executes from the top, but completed steps return their **recorded results** from the log instead of re-running; real execution resumes only past the end of history. Fan-in is a read over the ordered log ("is there a completion for every scheduled task"), decided by a **single evaluator** — so no concurrent-counter race. Requires **deterministic** workflow code (no clocks/random outside activities).

### Reach for it when

- **Fan-out/fan-in**: spawn N tasks, run one step when all finish (transcode ladder → mark READY).
- Long-running, multi-step, must survive crashes (order saga, provisioning).
- Steps need **retries**, **timeouts**, **compensation** without hand-rolled state.
- You'd otherwise build a status table + counter + reaper by hand.

#### Avoid / careful when

- Single-step / fire-and-forget — a queue is enough.
- Ultra-low-latency synchronous paths (replay + persistence add overhead).
- **Activities are at-least-once** — the continuation is exactly-once, but activity code must be idempotent.

#### Two flavors

| Temporal       | your code + replay        |
|----------------|---------------------------|
| Step Functions | declarative JSON FSM      |
| Fan-in         | built-in primitive        |
| Guarantee      | exactly-once continuation |

**CAP / consistency.** The orchestrator's own durability comes from its backing store (Temporal: Cassandra/Postgres/MySQL; Step Functions: managed). What it *gives* you is exactly-once **continuation** — the workflow advances as if each step ran once — layered on at-least-once **activity execution**. The two are different; your activities still need idempotency.

**Interview line.** When I need to fan out N tasks and fire one step when all complete, I reach for an orchestrator instead of a hand-rolled counter-and-reaper. It persists every transition to a durable log and decides completion with a single reader over that ordered log, so the fan-in race is gone by construction. Continuation is exactly-once; I still make activities idempotent.

**Pushback / when it flips.** Don't reach for it for a single async job — that's a queue, and the replay/determinism model is real cognitive + operational cost. The mechanism to **name** even if you hand-roll: atomic **DECR** for "am I last," a guarded status transition for idempotent completion, and a reaper for stalls.

## 12 · ZooKeeper / etcd

*Strongly-consistent coordination for small metadata: leader election, distributed locks, config, membership. Not a data store — a consensus-backed source of truth for "who's the leader" and "who's alive." (etcd is the modern, Raft-based equivalent.)*

**Mechanism.** A replicated in-memory tree of **znodes** (etcd: a flat key space), kept consistent across a small ensemble by an atomic-broadcast/consensus protocol — **ZAB** for ZooKeeper, **Raft** for etcd. Writes go through a leader and commit only on a **quorum** (majority), so an ensemble of 2f+1 tolerates f failures (5 nodes → survive 2). Three primitives carry all the use cases: **ephemeral** znodes vanish when a client's session heartbeat lapses (→ liveness/membership); **watches** notify clients of changes (→ config push, leader-change notification); **sequential** znodes give globally ordered names (→ fair locks and leader election).



## Reach for it when

- **Leader election** / single-writer coordination — who owns this shard, who runs the singleton cron.
- **Distributed locks / barriers** that must be *correct* (ephemeral + sequential znodes).
- **Service discovery / membership** — ephemeral nodes as a live registry.
- **Config that must be consistent** and pushed to watchers on change.

## The distinctions to say

- **Not a database**: kilobytes per znode, whole tree in memory. Store *pointers and coordination state*, never bulk data.
- **Consistency over availability**: writes need quorum, so a minority partition **can't write** — CP by design. That's the point; you want the lock *wrong-proof*, not always-available.
- **Redis lock ≠ this**: **SET NX** + fencing is a best-effort **lease** (safe with a fencing token, but availability-first); ZooKeeper/etcd give **consensus-backed** correctness. Name the difference when a double-holder is a correctness bug.
- **Often invisible**: it's the thing *under* other systems — Kafka (historically), HBase, and etcd under Kubernetes — so you frequently use it via another system rather than directly.

## Numbers to anchor

| Ensemble    | 3 or 5 ( $2f+1$ , tolerates $f$ ) |
|-------------|-----------------------------------|
| Data size   | KB per znode (metadata only)      |
| Writes      | quorum consensus (low throughput) |
| Consistency | linearizable writes (CP)          |

**CAP / consistency.** Strongly consistent / **CP**: linearizable writes via quorum; reads are sequentially consistent and can lag slightly (use a **sync** /quorum read for linearizable reads). It deliberately sacrifices availability in the minority partition — the ensemble stops accepting writes rather than risk split-brain, which is exactly what you want backing a lock or leader election.

**Interview line.** For leader election, locks, or membership I want consensus-backed coordination — ZooKeeper or etcd — not a Redis lock. Ephemeral znodes give me liveness, sequential znodes give ordered fair locks, and quorum writes make it CP so the lock can't split-brain. I keep only kilobytes of coordination metadata there; the real data lives elsewhere.

**Pushback / when it flips.** Reaching for ZooKeeper *directly* is increasingly rare — managed systems hide it, and **etcd** (Raft, behind Kubernetes) is the modern default. For a best-effort lock where an occasional double-run is tolerable, a Redis lease is simpler and cheaper; escalate to consensus only when a double-leader is a correctness bug, not a nuisance. And never put throughput or bulk data on it — every write is a consensus round.

---

## 13 · Elasticsearch

*An inverted index for full-text and faceted search. A secondary read model you feed from your primary store — not your source of truth.*

**Mechanism.** Inverted index (Lucene) + distributed shards. Text is analyzed (tokenized, lowercased, stemmed) into terms; the inverted index maps term → documents, so full-text queries are fast. Relevance ranked by **BM25** . Indexes shard + replicate for scale and HA. **Near-real-time**, not immediate — a refresh interval (default ~1s) gates visibility. Async-fed from your primary DB via CDC or dual-write.

**Reach for it when**

- **Full-text search** with relevance ranking (search bars, docs, logs).
- **Faceted / filtered** search — aggregations across many fields (e-commerce filters).
- Log / observability search at scale (the ELK stack).
- Hybrid with vectors for semantic + keyword.

**Avoid / careful when**

- As a **primary store** — it's a derived index; rebuild from truth. Don't put data only here.
- You need **strong consistency / transactions** — near-real-time and no ACID.
- Simple exact lookups — a DB index is cheaper and simpler.
- Keeping it in sync is the real cost — CDC pipeline + reconciliation.

**Numbers to anchor**

| Visibility | near-real-time (~1s) |
|------------|----------------------|
| Ranking    | BM25                 |
| Role       | derived read model   |

**CAP / consistency.** AP-leaning and eventually consistent with your primary by construction (async-fed, ~1s refresh). Treat divergence as normal: design the sync pipeline, a reconciliation path, and the ability to rebuild the index from the source of truth at any time.

**Interview line.** Full-text and faceted search go to Elasticsearch as a secondary read model fed from my primary store via CDC — the inverted index and BM25 give relevance ranking a B-tree can't. It's near-real-time and not my source of truth, so I design the sync pipeline and a reconciliation path, and I can rebuild the index from the primary at any time.

**Pushback / when it flips.** The trap is treating it as a database. It's a **derived index** — the hard part is the sync pipeline and its consistency, not the query. For semantic search, **hybrid BM25 + vector** usually beats either alone. For plain exact-match, a Postgres index is simpler and consistent.

**14 · Decision matrix — workload → pick**

*Collapse the thirteen stores into one glance. Match the workload on the left to the default pick; the "because" is the one-line justification.*

| Workload                                                | Default pick                       | Because                                                                        |
|---------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------------|
| Transactions, joins, ad-hoc queries                     | <b>PostgreSQL</b>                  | ACID + query flexibility; the default until something breaks                   |
| Keyed lookups, global reads, huge scale (self-run)      | <b>Cassandra</b>                   | auto-rebalance + multi-region writes; you don't need joins                     |
| Keyed lookups at scale, serverless / zero-ops           | <b>DynamoDB</b>                    | Dynamo-lineage auto-shard, managed, per-request cost; design the partition key |
| Write firehose / > ~50k writes-s to one table           | <b>Cassandra / DynamoDB / TSDB</b> | LSM absorbs append-heavy volume one Postgres primary can't                     |
| Cache / counter / lock / TTL'd ephemeral state          | <b>Redis</b>                       | in-memory, atomic ops, microsecond latency                                     |
| Decouple + replay + multiple consumers                  | <b>Kafka</b>                       | retained ordered log, rewindable, fan-out                                      |
| Windowed aggregation / stream join / sessionize         | <b>Flink</b>                       | stateful stream compute, event-time watermarks, exactly-once state             |
| Background jobs with retry + DLQ                        | <b>Queue (SQS/RabbitMQ)</b>        | per-message ack/redelivery; no replay needed                                   |
| Large binaries (video, images, uploads)                 | <b>S3 / Blob</b>                   | infinite cheap durable bytes; keep the pointer in the DB                       |
| Global static / hot read-heavy content                  | <b>CDN</b>                         | serve from the edge; collapse read fan-out                                     |
| Semantic search / RAG retrieval                         | <b>Vector DB</b>                   | ANN over embeddings; hybrid with BM25 for exact terms                          |
| Coordinate N tasks, fan-in, sagas                       | <b>Orchestrator</b>                | durable fan-out/fan-in, exactly-once continuation                              |
| Leader election / lock / membership / consistent config | <b>ZooKeeper / etcd</b>            | consensus-backed coordination (CP); not a data store                           |
| Full-text / faceted search                              | <b>Elasticsearch</b>               | inverted index + BM25; a derived read model, not truth                         |

**The meta-rule.** Most systems are **Postgres + one or two specialists**: Postgres as the transactional source of truth, plus a cache (Redis), a blob store (S3), a CDN, and *maybe* a search or vector index fed from Postgres via CDC. Reach for Cassandra/DynamoDB/Kafka/Flink/an orchestrator only when a named requirement — a specific hot table past ~15–50k writes/s, global write scale, replay/fan-out, real-time stream compute, durable multi-step coordination — forces it. Naming the *one* store as truth and treating the rest as derived is the coherence signal.

#### Cross-cutting principles that reappear everywhere

- **Idempotency** is non-negotiable anywhere delivery is at-least-once (Kafka, queues, orchestrator activities, Flink sinks, retried writes).
- **One source of truth**; everything else (cache, search index, vector store, denormalized copy) is derived and must be rebuildable + reconciled.

- **Reads tolerate staleness; writes must be correct** — the asymmetry behind CDNs, replicas, caches, and search indexes.
- **Move a guarantee out of the database only when arrival rate makes the DB physically unable to provide it** — the same rule for sharding (~15–50k writes/s to one table), Redis holds, and counter offloading (~500–1k/s to one hot row).
- **Consensus (ZooKeeper/etcd) for correctness-critical coordination; a lease (Redis) when an occasional double is tolerable** — pick the strength of guarantee the failure demands.